Quiz - Divide and Conquer

Fall 2026 Advanced Algorithms (SE-4230-001). Correct answers are marked with *****.

Question 1

Solve the following recurrence relations. Use the master theorem or draw the recursion tree.

Quick Rule: use gg = a / (b^d)

For T(n) = aT(n/b) + O(n^d)	What it means	Answer
gg = 1	**levels are the same**	**Theta(n^d log n)**
gg < 1	**root dominates**	**Theta(n^d)**
gg > 1	**leaves dominate**	**Theta(n^(log_b a))**

Recurrence	gg math	Case	Correct answer
T(n) = 2T(n/2) + O(n)	2/(2^1) = 1	levels same	* Theta(n log n)
T(n) = 2T(n/2) + O(1)	2/(2^0) = 2	leaves dominate	* Theta(n)
T(n) = T(n/2) + O(1)	1/(2^0) = 1	levels same	* Theta(log n)
T(n) = T(n/2) + O(n)	1/(2^1) = 1/2	root dominates	* Theta(n)
T(n) = 4T(n/2) + O(n)	4/(2^1) = 2	leaves dominate	* Theta(n^2)
T(n) = 4T(n/2) + O(n^2)	4/(2^2) = 1	levels same	* Theta(n^2 log n)
T(n) = T(n-1) + O(n)	Master theorem N/A	unroll sum	* Theta(n^2)
T(n) = T(n-1) + O(1)	Master theorem N/A	unroll count	* Theta(n)

For the two T(n-1) recurrences, Master Theorem does not apply because the subproblem size is n-1, not n/b. Unroll them instead:

```
T(n) = T(n-1) + O(n)
= n + (n-1) + (n-2) + ... + 1 = Theta(n^2)
```

Correct answer: * Theta(n^2)

```
T(n) = T(n-1) + O(1)
= 1 + 1 + ... + 1 for n levels = Theta(n)
```

Correct answer: * Theta(n)

Question 2

Solve the recurrence: F(n) = 4F(n/2) + n^2

```
a = 4, b = 2, d = 2, so gg = 4/(2^2) = 1. Levels are the same.
```

* Theta(n^2 log n)

Theta(n^3)

Theta(2^n)

Theta(n^(1/2))

Theta(n log n)

Question 3

Match each Master Theorem case to the bound it gives for T(n) = aT(n/b) + O(n^d).

* Root dominates: d > log_b(a), equivalently gg < 1 -> Theta(n^d)

* Every level costs the same: d = log_b(a), equivalently gg = 1 -> Theta(n^d log n)

* Leaves dominate: d < log_b(a), equivalently gg > 1 -> Theta(n^(log_b a))

Question 4

A divide-and-conquer algorithm splits the input into 3 subproblems of size n/3, solves 2 of the 3 recursively, and combines the results in O(n) time.

```
Recurrence: T(n) = 2T(n/3) + O(n). Here a = 2, b = 3, d = 1, so gg = 2/(3^1) = 2/3 < 1. Root
dominates.
```

Theta(n log n), because the combination step is linear.

* Theta(n), because only 2/3 of the work is done at each level.

Theta(n^(log_3 2)) ~= Theta(n^0.63), because the leaves dominate.

Theta(n^2), because two recursive calls each see n/3 of the input.

Question 5

Karatsuba's algorithm multiplies two n-digit numbers significantly faster than the grade-school algorithm. Select all true statements.

Reducing recursive calls from 4 to 3 changes which case of the master theorem applies and yields a better asymptotic bound.

* Karatsuba runs in Theta(n^(log_2 3)) ~= Theta(n^1.585), which is asymptotically faster than the Theta(n^2) grade-school algorithm.

Karatsuba's algorithm requires Theta(n^2) extra space.

* The grade-school algorithm uses 4 multiplications of (n/2)-digit numbers at each level of recursion; Karatsuba uses only 3, using a clever algebraic identity.

The savings from Karatsuba come entirely from reducing the combination/addition step.

Math check: grade-school split is 4T(n/2) + O(n), so gg = 4/2 = 2 and leaves give Theta(n^2). Karatsuba is 3T(n/2) + O(n), so gg = 3/2 > 1 and leaves give Theta(n^(log_2 3)). Both are leaves-dominated; Karatsuba improves the exponent by reducing recursive multiplications.

Question 6

Select all true statements about divide-and-conquer.

If a divide-and-conquer algorithm's sub-problems overlap, memoization cannot help.

* The same recursion tree analysis used to derive the master theorem can be applied to recurrences the theorem doesn't neatly cover.

* Divide-and-conquer decomposes a problem into sub-problems of the same type that typically do not overlap.

* Merge sort's correctness relies on the fact that merging two sorted halves into a sorted whole can be done in O(n) time.

Divide-and-conquer always gives a sub-quadratic algorithm.

* The recurrence T(n) = aT(n/b) + f(n) captures the runtime of a divide-and-conquer algorithm that makes a recursive calls on inputs of size n/b with O(f(n)) overhead.

Wording note: for merge sort, the correctness part is that the merge step correctly combines two sorted halves into one sorted whole; the O(n) part is the runtime of that combine step.